

TECHNICAL DOCUMENTATION

Trao AI Travel Planner

Phase 1 — Monorepo Setup & Project Initialization

PHASE	COMMIT	DATE	AUTHOR
01 / 12	feat: initialize monorepo	June 2026	Abhi

Phase 1 establishes the **monorepo foundation** — a single root directory containing both the **backend** (Node.js + Express) and **frontend** (React + Vite) as separate workspaces. No application logic is written in this phase; the goal is purely to get the correct folder structure, dependencies, and tooling in place so every subsequent phase has a stable base to build on.



Monorepo Pattern

Both backend and frontend live in one Git repository, making it easy to share code, run both servers, and deploy from a single source of truth.



Vite over CRA

Vite is chosen for the frontend because it uses native ES modules for near-instant HMR (Hot Module Replacement) during development — much faster than Create React App.



ES Modules Backend

"type": "module" in package.json makes the backend use import/export syntax (ESM) instead of require/module.exports (CJS), keeping syntax consistent across the whole project.



No Nodemon

Node.js 18+ has a built-in --watch flag. We use --watch-path=./ with --env-file=.env to auto-restart and load environment variables without any extra dependencies.

After Phase 1 completes, the project tree looks like this. Every folder is intentionally empty for now — they will be filled in subsequent phases.

```
traoproject/           ← Git repository root
├── backend/           ← All server-side code
│   ├── config/        ← DB connection (Phase 2)
│   ├── middleware/     ← JWT auth (Phase 3)
│   ├── models/         ← Mongoose schemas (Phase 2)
│   ├── controllers/    ← Business logic (Phase 3+)
│   ├── routes/         ← Express routes (Phase 3+)
│   ├── package.json    ← Backend deps + scripts
│   └── server.js       ← Entry point (Phase 3)
└── frontend/          ← All client-side code
    ├── src/
    │   ├── pages/      ← Login, Register, Dashboard
    │   ├── components/ ← Reusable UI components
    │   ├── utils/      ← Axios instance + helpers
    │   ├── App.jsx      ← Root component + routing
    │   ├── main.jsx     ← React DOM entry point
    │   └── index.css    ← Tailwind directives
    ├── tailwind.config.js ← Tailwind CSS config
    ├── postcss.config.js ← PostCSS pipeline config
    └── package.json     ← Frontend deps + Vite scripts
```

This is the most important configuration file for the backend. Every field is deliberately chosen. Here is the complete annotated file:

```
backend/package.json JSON

{
  // "name" - identifies this npm package; used in logs and npm registry
  "name": "server",

  // "version" - semantic versioning; starts at 1.0.0 for a new project
  "version": "1.0.0",

  // "type": "module" - CRITICAL! Tells Node.js to use ES Module syntax.
  // This means every .js file in this project must use:
  //   import X from 'y'      ← instead of: const X = require('y')
  //   export default X      ← instead of: module.exports = X
  // Without this, Node defaults to CommonJS (require) syntax.
  "type": "module",

  // "main" - the entry point file Node.js runs when you start the server
  "main": "server.js",

  "scripts": {
    // "test" - placeholder; no tests configured yet in Phase 1
    "test": "echo \"Error: no test specified\" && exit 1",

    // "dev" - runs the server in DEVELOPMENT mode with:
    //   --env-file=.env      reads .env file and injects variables into process.env
    //                       (replaces the 'dotenv' npm package - built into Node 20.6+)
    //   --watch-path=./      watches ALL files in the backend folder for changes
    //                       and automatically restarts the server (replaces 'nodemon')
    //   server.js            the file to execute
    "dev": "node --env-file=.env --watch-path=./ server.js",

    // "start" - runs the server in PRODUCTION mode:
    //   No --watch-path because production servers should NOT auto-restart on file change
    //   --env-file=.env still loads environment variables from the .env file
    "start": "node --env-file=.env server.js"
  },

  "author": "Abhi",    // project owner name
  "license": "MIT",    // open source license type

  "dependencies": {
    // bcryptjs - hashes passwords before storing in database.
  }
}
```

```

//  Never store plain-text passwords. bcrypt adds a "salt" (random data)
//  so even identical passwords produce different hashes.
"bcryptjs": "^3.0.3",

//  cors – Cross-Origin Resource Sharing middleware.
//  Browsers block requests from a different domain (e.g., localhost:5173 → localhost:5000).
//  cors() middleware adds HTTP headers that tell browsers: "this is allowed."
"cors": "^2.8.6",

//  express – the web framework. Handles HTTP routing, middleware, and responses.
//  express@5 (latest) uses async/await error handling natively.
"express": "^5.2.1",

//  jsonwebtoken – creates and verifies JSON Web Tokens for authentication.
//  After login, the server signs a JWT with the user's ID.
//  The frontend sends this token with every request to prove identity.
"jsonwebtoken": "^9.0.3",

//  mongoose – ODM (Object Data Modeling) library for MongoDB.
//  Provides schemas, validation, and query helpers so we work with
//  JavaScript objects instead of raw MongoDB documents.
"mongoose": "^9.7.1"
}

// NOTE: 'dotenv' and 'nodemon' are intentionally NOT listed here.
// dotenv is replaced by Node's built-in --env-file flag (Node 20.6+).
// nodemon is replaced by Node's built-in --watch flag (Node 18+).
}

```

WHY "TYPE": "MODULE" MATTERS

Without `"type": "module"`, if you write `import express from 'express'` in a `.js` file, Node.js will throw:
SyntaxError: Cannot use import statement in a module.

With it set, Node treats every `.js` file in this folder as an ES Module. This means you must also use `import` for all built-in Node modules: `import { createRequire } from 'module'`, etc.

Tailwind CSS scans your source files to find which CSS utility classes you actually use, then generates a minimal CSS bundle containing only those classes. The content array tells Tailwind where to look.

frontend/tailwind.config.js

JavaScript

```
// JSDoc type hint – gives your editor IntelliSense for the config object
/** @type {import('tailwindcss').Config} */

// ES Module export (works because frontend uses Vite which supports ESM)
export default {

  // "content" – array of glob patterns pointing to files that USE Tailwind classes.
  // Tailwind reads these files at build time and tree-shakes unused styles.
  // IMPORTANT: If a file is not in this list, its classes won't be in the final CSS.
  content: [
    "./index.html", // ← root HTML file (Vite's entry point)
    "./src/**/*.{js,ts,jsx,tsx}" // ← ALL files inside src/ with these extensions
    // ** means any subfolder depth
    // {js,ts,jsx,tsx} is a brace expansion matching any of those ex
  ],

  theme: {
    // "extend" – add custom values WITHOUT removing Tailwind's defaults.
    // For example: extend.colors lets you add brand colors while keeping all default colors.
    // In Phase 1 this is empty – we'll add custom tokens in later phases if needed.
    extend: {},
  },

  // "plugins" – official or community Tailwind plugins (e.g., @tailwindcss/forms).
  // Empty for now – no extra plugins needed in this project.
  plugins: [],
}
```

This is the global CSS entry point imported in `main.jsx`. In Phase 1 we replace the default Vite CSS with just three Tailwind directives. These directives are special Tailwind-specific instructions processed by PostCSS at build time.

frontend/src/index.css

CSS

```
/* @tailwind base
   Injects Tailwind's "Preflight" – a CSS reset based on modern-normalize.
   This normalises browser inconsistencies: removes default margins on headings,
   sets box-sizing: border-box everywhere, normalises form element styling, etc.
   looks different in Chrome vs Firefox vs Safari. */
@tailwind base;

/* @tailwind components
   Injects any component classes defined by Tailwind plugins or your own
   @layer components {} rules. In Phase 1 there are none.
   Later you can write: .btn { @apply px-4 py-2 rounded bg-indigo-600; }
   and it gets injected here. */
@tailwind components;

/* @tailwind utilities
   Injects ALL utility classes: flex, grid, p-4, text-slate-800, etc.
   This is the core of Tailwind. Vite + PostCSS processes this directive
   and outputs only the classes found in your content files (tree-shaking). */
@tailwind utilities;
```

Without this, a

🔍 HOW THE CSS PIPELINE WORKS

1. Vite starts the dev server and processes `main.jsx` → imports `index.css`
2. PostCSS intercepts the CSS and runs the Tailwind plugin
3. Tailwind scans all files matching content globs to find class names
4. Only the found classes are compiled into actual CSS and injected into the browser
5. Result: tiny CSS file with zero unused styles

Backend Dependencies

PACKAGE	VERSION	PURPOSE	USED IN PHASE
<code>express</code>	<code>^5.2.1</code>	HTTP server framework — handles routes, middleware, requests/responses	Phase 3
<code>mongoose</code>	<code>^9.7.1</code>	MongoDB ODM — define schemas, validate data, run queries as JS objects	Phase 2
<code>jsonwebtoken</code>	<code>^9.0.3</code>	Sign and verify JWT tokens for stateless authentication	Phase 3
<code>bcryptjs</code>	<code>^3.0.3</code>	Hash passwords with salt before storing; compare hash on login	Phase 3
<code>cors</code>	<code>^2.8.6</code>	Allow cross-origin requests from the React frontend (different port)	Phase 3

Frontend Dependencies

PACKAGE	VERSION	PURPOSE	USED IN PHASE
<code>react</code>	<code>^19.x</code>	UI library — component model, hooks, virtual DOM	Phase 8
<code>react-dom</code>	<code>^19.x</code>	Renders React components into the real browser DOM	Phase 8
<code>react-router-dom</code>	<code>^7.x</code>	Client-side routing — navigate between pages without full reload	Phase 8
<code>axios</code>	<code>^1.x</code>	HTTP client — makes API calls to backend with interceptors for auth	Phase 8
<code>tailwindcss</code>	<code>^3.x</code>	Utility-first CSS framework — all styling via class names in JSX	Phase 8+
<code>postcss</code>	<code>^8.x</code>	CSS transformation pipeline — processes Tailwind directives	Phase 8+
<code>autoprefixer</code>	<code>^10.x</code>	Adds vendor prefixes (-webkit-, -moz-) for browser compatibility	Phase 8+

PACKAGE	VERSION	PURPOSE	USED IN PHASE
vite	^6.x	Build tool and dev server — fast HMR, native ESM, optimized builds	All Frontend

Every command run during Phase 1, with an explanation of what it does:

Terminal (PowerShell)

Shell

```
# — BACKEND SETUP —  
  
# Creates backend/, config/, middleware/, models/, controllers/, routes/ folders  
# -Force: doesn't error if folder already exists  
New-Item -ItemType Directory -Force -Path "backend", "backend\config", ...  
  
# Generates backend/package.json with default values (-y skips all questions)  
cd backend && npm init -y  
  
# Installs 5 production packages into backend/node_modules/  
# Also creates package-lock.json (locks exact dependency versions for reproducibility)  
npm install express mongoose jsonwebtoken bcryptjs cors  
  
# Removes nodemon and dotenv that were previously installed  
# We replaced them with Node.js built-in --watch and --env-file flags  
npm uninstall nodemon dotenv  
  
# — FRONTEND SETUP —  
  
# Scaffolds a complete React + Vite project in the frontend/ directory  
# --template react: uses JavaScript (not TypeScript) React template  
# This creates: src/, public/, index.html, vite.config.js, package.json, etc.  
npm create vite@latest frontend -- --template react  
  
# Installs all packages listed in frontend/package.json (react, react-dom, vite)  
cd frontend && npm install  
  
# Installs Axios (HTTP client) and React Router DOM (client-side routing)  
npm install axios react-router-dom  
  
# Installs Tailwind CSS v3, PostCSS, and Autoprefixer as dev dependencies  
# (-D is shorthand for --save-dev: these are build tools, not runtime dependencies)  
npm install -D tailwindcss@3 postcss autoprefixer  
  
# Initialises Tailwind by generating two config files:  
#   tailwind.config.js — where you configure content paths, theme, plugins  
#   postcss.config.js — tells PostCSS to use tailwindcss and autoprefixer plugins  
# -p flag: generate postcss.config.js at the same time  
npx tailwindcss init -p
```

Auto-generated by `npx tailwindcss init -p`. PostCSS is a CSS processor that runs plugins. This config tells PostCSS which plugins to use when processing CSS files.

frontend/postcss.config.js

JavaScript

```
// PostCSS configuration – defines which CSS processing plugins to use
export default {
  plugins: [

    // tailwindcss – the main Tailwind plugin.
    // PostCSS runs this first: it reads tailwind.config.js, scans content files,
    // and replaces @tailwind directives with actual generated CSS classes.
    tailwindcss: {},

    // autoprefixer – runs AFTER tailwindcss.
    // Reads generated CSS and adds vendor-prefixed versions of properties
    // that some browsers still need, e.g.:
    //   display: flex → also adds display: -webkit-flex
    //   transform: ... → also adds -webkit-transform: ...
    // This ensures compatibility with older Safari / iOS browsers.
    autoprefixer: {},
  ],
}
```

Phase 1 Completion Checklist

- ✓ `backend/` directory created with all subdirectories (config, middleware, models, controllers, routes)
- ✓ `backend/package.json` has "type": "module", correct scripts with `--env-file` and `--watch-path`
- ✓ `express`, `mongoose`, `jsonwebtoken`, `bcryptjs`, `cors` installed in backend
- ✓ `nodemon` and `dotenv removed` — replaced by Node.js built-in equivalents
- ✓ `frontend/` scaffolded with `npm create vite@latest --template react`
- ✓ `axios` and `react-router-dom` installed in frontend
- ✓ `tailwindcss@3`, `postcss`, `autoprefixer` installed as dev dependencies
- ✓ `tailwind.config.js` configured with correct content glob paths
- ✓ `postcss.config.js` generated (auto) with `tailwindcss` and `autoprefixer` plugins
- ✓ `src/index.css` replaced with Tailwind `@tailwind base/components/utilities` directives
- ✓ `src/pages/`, `src/components/`, `src/utils/` empty directories created

 GIT COMMIT FOR THIS PHASE

```
git commit -m "feat: initialize monorepo folder structure with backend and frontend"
```